

Minimal IoT Dashboard

Teammate Guide: Understanding SRS, SAD & Test Plan Documents

1. Welcome to the Team Project Documentation Guide!

This guide explains our project documentation (**SRS**, **SAD**, and **Test Plan**) in simple, straightforward language. As a team building the **Minimal IoT Dashboard** over this semester, these 3 documents serve as our architectural blueprint, developer specifications, and quality verification standard.

2. What are the 3 Core Documents?

A. SRS (Software Requirements Specification) — WHAT We Are Building

The SRS defines every single feature and functional expectation for our IoT dashboard. It answers: *What features must be delivered? What response times are expected? How should errors be handled?* Each requirement has a unique ID (e.g. IOT-FR-001) that maps directly to your code.

B. SAD (Software Architecture & Design) — HOW We Build It

The SAD outlines the technical design, microservice components, technology stack, database schemas, API contracts (REST & WebSockets), sequence diagrams, and security threat model (STRIDE). It ensures all 4 components connect seamlessly.

C. Test Plan (Software Test Plan / STP) — HOW We Prove It Works

The Test Plan specifies how we verify our code before release. It breaks down testing into Unit Tests, Integration Tests, System Tests, and Security/Performance Tests with concrete Pass/Fail test cases (e.g. TC-UI-01, TC-INGEST-01).

3. Team Role Cheat Sheet & Deliverable Mapping

Here is how each team member's role maps directly into the 3 documents:

Member & Role	What You Focus On in SRS	What You Focus On in SAD	Your Test Cases in Test Plan
Member 1 Frontend Lead	- Interactive live charts - Telemetry status cards - Log viewer with CSV export - WebSocket auto-reconnect	- Component state management - Chart.js / ECharts integration - Client CSV/JSON export logic - Reconnect exponential backoff	TC-UI-01: Live chart updates TC-UI-02: CSV log export TC-UI-03: Auto-reconnect resilience
Member 2 Backend Lead	- REST API endpoints - WebSocket/SSE broadcasting - Dynamic threshold rule engine - API rate limiting & Auth	- REST API specifications - WebSocket stream architecture - Dynamic threshold evaluator - JWT auth & rate limiters	TC-INGEST-01: Valid payload ingest TC-WS-01: Sub-200ms WS broadcast TC-SEC-01: Rate limit HTTP 429
Member 3 Database Lead	- Time-series logging - Continuous rollups (1m, 1h, 1d) - Retention policy pruning - Multi-sensor telemetry simulator	- TimescaleDB hypertable schema - Materialized view SQL rollups - Diurnal curve generator logic - Humidity noise injection	TC-SIM-01: Simulator curve output TC-DB-01: Hourly rollup accuracy TC-DB-02: Retention policy pruning
Member 4 DevOps Lead	- Z-Score & Isolation Forest - Multi-channel alert worker - Unified docker-compose.yml - CI/CD pipeline & docs	- Z-Score math & rolling stats - Isolation Forest ML feature vector - STRIDE threat mitigations - Docker microservices diagram	TC-ANOM-01: Z-Score spike alert TC-ALERT-01: Discord webhook dispatch TC-DOCKER-01: 5-container startup

4. How We Collaborate & Commit Throughout the Semester

To maintain an active, high-quality GitHub repository and score top marks, we follow these best practices:

- **Incremental Daily Commits:** Commit small, modular changes daily rather than giant dumps.

- **Traceability:** Mention Requirement IDs (e.g. IOT-FR-030) or Test Case IDs (e.g. TC-UI-01) in your commit messages.
- **Documentation First:** Keep docs/SRS.md, docs/SAD.md, and docs/TEST_PLAN.md updated as our code evolves.
- **Docker Integration:** Ensure every new feature works inside our unified docker-compose.yml environment.